

L02- Deploying an AI Model on a Simulated Edge Device

Chosen Approach: Option A – Conceptual

Part 1: Conceptual Setup of Development Environment

Python Installation and Configuration

Step 1: Downloading Python

1. Navigate to the official Python website: <https://www.python.org>
2. Select the **Downloads** section.
3. Choose the latest stable version of Python compatible with your operating system (Windows, macOS, or Linux).
 - a. For AI and TensorFlow compatibility, Python **3.9–3.11** is commonly recommended.
4. Click the appropriate installer (e.g., *Windows Installer (64-bit)*).

Step 2: Installing Python

1. Launch the downloaded installer.
2. **Important:** Check the box labeled “**Add Python to PATH**” before proceeding.
3. Select **Install Now** (recommended for most users).
4. Allow the installation to complete.

Step 3: Verifying the Installation

After installation:

1. Open a command prompt (Windows) or terminal (macOS/Linux).
2. Enter the following command:

L02- Deploying an AI Model on a Simulated Edge Device

```
python -version
```

3. If Python is installed correctly, the system will display the installed version number.

Step 4: Understanding PATH Configuration

- The **PATH environment variable** allows the operating system to locate Python from any command-line location.
- If Python was not added to PATH during installation, it must be added manually through system environment variables.
- Proper PATH configuration ensures compatibility with tools such as:
 - TensorFlow
 - TensorFlow Lite Converter
 - Python virtual environments

TensorFlow and TensorFlow Lite Installation

Step 1: Verify Python and pip Installation

Before installing TensorFlow:

1. Open a command prompt or terminal.
2. Confirm Python is installed:

```
python --version
```

3. Confirm the Python package manager (pip) is available:

```
pip --version
```

L02- Deploying an AI Model on a Simulated Edge Device

Step 2: Installing TensorFlow

TensorFlow is installed using the `pip` package manager.

Command:

```
pip install tensorflow
```

- This command downloads and installs TensorFlow along with all required dependencies.
- The installation may take several minutes depending on system performance and internet speed.

Step 3: Installing TensorFlow Lite

TensorFlow Lite functionality is included within the TensorFlow package; however, additional tools may be installed for development and conversion.

Optional TensorFlow Lite Support Tools:

```
pip install tf-lite-support
```

- TensorFlow Lite converter utilities are accessed through TensorFlow itself.
- The `tf-lite-support` package provides metadata handling and model validation tools for edge deployment.

Step 4: Verifying TensorFlow Installation

After installation:

L02- Deploying an AI Model on a Simulated Edge Device

1. Launch Python:

```
python
```

2. Import TensorFlow:

```
import tensorflow as tf  
print(tf.__version__)
```

3. Successful execution confirms TensorFlow and TensorFlow Lite are installed correctly.

Jupyter Notebook Installation

Overview

Jupyter Notebook is an interactive development environment commonly used for AI and data science projects. It allows developers to write Python code, visualize outputs, and document results in a single interface.

Step 1: Installing Jupyter Notebook

Jupyter Notebook is installed using pip.

Command:

```
pip install notebook
```

- This command installs Jupyter Notebook and its required dependencies.

L02- Deploying an AI Model on a Simulated Edge Device

Step 2: Launching Jupyter Notebook

Once installed:

1. Open a command prompt or terminal.
2. Navigate to the desired project directory (optional).
3. Launch Jupyter Notebook:

```
jupyter notebook
```

Step 3: Accessing the Jupyter Interface

- A web browser will automatically open displaying the Jupyter Notebook dashboard.
- Users can create new notebooks, manage files, and execute Python code interactively.

Part 2: Conceptual Creation and Training of an AI Model

Design of a Neural Network Model for the MNIST Dataset

Model Architecture

Input Layer

- **Input shape:** (28, 28)
- Represents grayscale pixel values of handwritten digits.
- Images are flattened before being passed into dense layers.

Hidden Layers

L02- Deploying an AI Model on a Simulated Edge Device

1. Flatten Layer

- a. Converts the 2D image input into a 1D array of 784 values.
- b. Required for compatibility with fully connected layers.

2. First Dense Layer

- a. **Number of neurons:** 128
- b. **Activation function:** ReLU (Rectified Linear Unit)
- c. Purpose: Extracts meaningful features and introduces non-linearity.

3. Second Dense Layer

- a. **Number of neurons:** 64
- b. **Activation function:** ReLU
- c. Purpose: Refines feature representations for improved classification.

Output Layer

- **Number of neurons:** 10
- **Activation function:** Softmax
- Purpose: Outputs a probability distribution across the 10 digit classes (0–9).

Summary of Model Architecture

Component	Description
Input Shape	28 × 28 grayscale images
Flatten Layer	Converts image to 1D vector
Dense Layer 1	128 neurons, ReLU
Dense Layer 2	64 neurons, ReLU
Output Layer	10 neurons, Softmax

L02- Deploying an AI Model on a Simulated Edge Device

Data Loading and Preprocessing Steps

Loading the MNIST Dataset

- The MNIST dataset is available directly through Keras datasets.
- It includes:
 - **60,000 training images**
 - **10,000 testing images**
- The dataset is automatically split into training and test sets upon loading.

Data Preprocessing

Normalization

- Pixel values range from **0 to 255**.
- To improve training efficiency, values are normalized to a **0–1 range** by dividing each pixel value by 255.
- Normalization helps stabilize gradient updates and accelerates convergence.

Data Type Conversion

- Image data is converted to floating-point format to ensure compatibility with neural network computations.

Preprocessing Summary

- Scale pixel values to [0, 1]
- Convert images to floating-point format
- Retain label values as integer class identifiers (0–9)

L02- Deploying an AI Model on a Simulated Edge Device

Model Training Process

Model Compilation

Before training, the model is compiled with the following components:

Optimizer

- **Adam (Adaptive Moment Estimation)**
- Selected for its efficiency and adaptive learning rate capabilities.

Loss Function

- **Sparse Categorical Crossentropy**
- Appropriate for multi-class classification when labels are integer-encoded.

Metrics

- **Accuracy**
- Used to evaluate model performance during training and validation.

Training the Model

- The training process involves feeding the preprocessed training data into the neural network.
- The model learns by adjusting weights to minimize the loss function.

Training Parameters

- **Number of epochs:** 5–10 (conceptually selected for balance between learning and overfitting)
- **Batch size:** Determined automatically or set to a standard value (e.g., 32)

During each epoch:

L02- Deploying an AI Model on a Simulated Edge Device

1. The model processes the entire training dataset.
2. Loss and accuracy metrics are computed.
3. Weights are updated based on optimization results.

Part 3: Conceptual Conversion and Saving of the Model

Model Conversion to TensorFlow Lite

Overview

After a neural network model has been trained using TensorFlow and Keras, it must be converted into a format suitable for deployment on edge devices. TensorFlow Lite (TFLite) is designed to provide a lightweight, efficient representation of trained models, optimized for environments with limited computing power, memory, and energy resources.

Purpose of TensorFlow Lite Conversion

The conversion process serves several key objectives:

- **Model Optimization:** Reduces model size and improves inference speed.
- **Edge Compatibility:** Ensures the model can run on mobile, embedded, and IoT devices.
- **Portability:** Produces a standalone `.tflite` file that can be deployed across multiple platforms.
- **Efficiency:** Enables faster inference with lower power consumption.

L02- Deploying an AI Model on a Simulated Edge Device

TensorFlow Lite Conversion Process

Step 1: Prepare the Trained Model

- The trained Keras model must be finalized after the training process.
- All layers, weights, and configurations are preserved during conversion.

Step 2: Initialize the TensorFlow Lite Converter

- TensorFlow provides a built-in converter utility that transforms a Keras model into TFLite format.
- The converter analyzes the model structure and parameters to generate an optimized representation.

Step 3: Apply Model Optimization (Optional)

Conceptually, optional optimizations may be applied during conversion:

- **Post-training quantization** to reduce model size
- **Weight compression** to improve performance
- **Operator optimization** for edge device compatibility

These optimizations help tailor the model for resource-constrained environments without significantly impacting accuracy.

Step 4: Convert the Model

- The converter transforms the original TensorFlow model into a **TensorFlow Lite FlatBuffer format**.

L02- Deploying an AI Model on a Simulated Edge Device

- The output is a binary representation that contains the optimized model architecture and trained weights.

Model Saving Process

Saving the TensorFlow Lite Model

Once conversion is complete:

- The resulting TensorFlow Lite model is saved as a file with a `.tflite` extension.
- This file represents the complete, deployable AI model.

Conceptual File Storage

- The `.tflite` file is typically stored in a project directory dedicated to deployment artifacts.
- The file can be transferred to edge devices or embedded into mobile applications.

Verification of the Saved Model

Conceptually, verification involves:

- Ensuring the file is successfully created.
- Confirming compatibility with TensorFlow Lite interpreters.
- Optionally testing inference using sample input data.

L02- Deploying an AI Model on a Simulated Edge Device

Part 4: Conceptual Deployment of the Model

Deployment Process on a Simulated Edge Device

Step 1: Writing a Deployment Script

Conceptually, a deployment script performs the following tasks:

1. Imports the TensorFlow Lite runtime or TensorFlow Lite interpreter.
2. Loads the saved `.tflite` model file from local storage.
3. Initializes an inference environment suitable for edge deployment.
4. Prepares input data in the required format.
5. Executes the model and retrieves outputs.

This script represents the interface between the AI model and the edge device environment.

Step 2: Setting Up the TensorFlow Lite Interpreter

The TensorFlow Lite interpreter is the core component responsible for executing the model.

Interpreter Setup Includes:

- Loading the `.tflite` model into memory.
- Creating an interpreter instance that manages model execution.
- Preparing internal data structures required for inference.

Step 3: Allocating Tensors

- Tensor allocation initializes memory buffers for:

L02- Deploying an AI Model on a Simulated Edge Device

- Input tensors (model inputs)
 - Output tensors (model predictions)
- Allocation ensures that all tensors are sized and structured according to the model's requirements.
- This step must be completed before inference can occur.

Model Testing Using Sample Input

Preparing Sample Input

To test the deployed model:

- A sample MNIST image is selected from the test dataset.
- The image undergoes the same preprocessing steps used during training:
 - Normalization to a 0–1 range
 - Reshaping to match the model's input dimensions

Consistency between training and deployment preprocessing is critical for accurate predictions.

Running Inference

- The preprocessed image is passed to the interpreter as input.
- The interpreter executes the model using the allocated tensors.
- The model processes the input and generates output probabilities.

Interpreting the Results

- The output tensor contains probability values for each digit class (0–9).

L02- Deploying an AI Model on a Simulated Edge Device

- The class with the highest probability is selected as the predicted digit.
- The prediction can be compared to the true label to assess correctness.

Reflective Journal

Introduction

This reflective journal summarizes my experience completing the conceptual assignment focused on designing, training, converting, and deploying an AI model using TensorFlow Lite. The assignment emphasized understanding the full AI lifecycle through research and conceptual design rather than hands-on implementation. This approach strengthened my foundational knowledge of edge AI deployment and prepared me for future practical applications.

L02- Deploying an AI Model on a Simulated Edge Device

Challenges Faced

One challenge during this assignment was translating technical, code-based processes into clear conceptual explanations. Without executing commands or scripts, it was necessary to rely on research and documentation to accurately describe environment setup, model training, and deployment steps.

Another difficulty involved understanding how TensorFlow and TensorFlow Lite interact. Research was required to clarify that TensorFlow Lite is integrated within TensorFlow and that additional tools enhance—but do not replace—the core framework. Additionally, conceptualizing deployment on edge devices required careful consideration of limitations such as reduced memory, processing power, and energy efficiency.

Learning Outcomes

This assignment improved my understanding of how AI models progress from development to deployment. I learned how Python-based environments support TensorFlow development and how neural network architecture choices—such as layer types, activation functions, and preprocessing—directly impact model performance.

I also gained a clear understanding of the purpose of TensorFlow Lite conversion, including model optimization and portability for edge devices. Learning how interpreters and tensor allocation enable inference in resource-constrained environments strengthened my understanding of real-world AI deployment.

Application of Knowledge

The knowledge gained from this assignment applies directly to real-world AI systems such as mobile applications, IoT devices, and embedded systems where on-device inference is

L02- Deploying an AI Model on a Simulated Edge Device

required. Conceptual planning and documentation are essential skills in professional AI development, especially when designing systems prior to implementation.

This conceptual foundation will also support the upcoming practical deployment assignment. By understanding the full workflow in advance, I will be better prepared to implement, test, and optimize AI models using TensorFlow Lite in a hands-on environment.

Conclusion

Overall, this assignment provided a comprehensive understanding of deploying AI models at the edge. By addressing challenges, reflecting on learning outcomes, and connecting concepts to real-world applications, I have developed the confidence and knowledge needed to successfully transition from conceptual design to practical implementation.